

CAR SERVICE BOOKING SYSTEM

Microservices Architecture — Capstone Project

Software Requirements Specification & Trainee Implementation Guide

Prepared for: Java Full Stack DSE Trainees

1. Project Overview

The **Car Service Booking System** is a modern, web-based microservices application designed to streamline the automotive maintenance process. It provides an end-to-end digital platform enabling customers to securely register, manage their vehicles, browse curated service catalogs, and seamlessly book service appointments online. For service center operators, the system offers a robust suite of tools to manage service offerings, track booking lifecycles, and monitor critical business metrics.

The application strictly enforces role-based workflows for two primary user types:

- **Customers:** Empowered to manage their personal profiles and vehicle fleets. They can browse active service packages, schedule appointments (preventing duplicate time slots), and track the real-time status of their ongoing service requests.
- **Administrators:** Responsible for overseeing the business logic. They manage service categories, toggle the availability of specific service packages, advance the status of customer bookings through a strict operational workflow (from PENDING to COMPLETED), and monitor system-wide activities via aggregated dashboard analytics.

2. System Architecture

The solution follows a Microservices Architecture consisting of Authentication Service, User Service, Service Catalog Service, and Booking Service, with communication managed through Spring Cloud Gateway and Spring Cloud Consul. All frontend requests must pass through the API Gateway, which is responsible for routing requests to the appropriate backend services.

2.1 Infrastructure Mapping

Service	Port	Database Schema
API Gateway	8080	None
Consul (Discovery)	8500	None
Auth Service	8081	auth_db
User Service	8082	user_db
Catalog Service	8083	catalog_db
Booking Service	8084	booking_db
Angular Frontend	4200	LocalStorage Session

2.2 Important Development Rule

API Gateway Communication: The Angular frontend must communicate only with `http://localhost:8080/api/...`. Do not hardcode direct microservice ports such as 8081, 8082, etc.

3. User Stories

The following ten user stories define the complete functional scope of the application. Trainees will implement **US05, US06, US07, and US10** from starter code.

The stories **US01, US02, US03, US04, US08, and US09** are pre-implemented and must remain functional without redesign.

US 01 – Authentication & Authorization

Priority: Must Have

Primary Microservice: Authentication Service

User Story Description

As a **Customer**, I want to securely register, and as a **User** (Customer or Administrator), I want to log in so that I can access the application based on my assigned role.

Acceptance Criteria

- Customers shall be able to register using Full Name, Email Address, and Password.
 - Email addresses shall be unique across the system.
 - Users shall be able to log in using their Email Address and Password.
 - Authentication shall be performed by validating the entered credentials directly against stored database records (without using Spring Security or JWTs).
 - User role shall be identified during successful login.
 - Role-based access shall be enforced throughout the application by passing HTTP headers (X-User-Role, X-User-Id).
 - Users shall be able to log out by clearing the active session in local storage.
 - Duplicate email registrations shall not be allowed.
 - Mandatory fields shall be validated before registration and login.
 - Appropriate success, validation, and error messages shall be displayed.
-

US 02 – Customer Profile Management

Priority: Must Have

Primary Microservice: User Service

User Story Description

As a **Customer**, I want to manage my profile information so that my personal details remain accurate for future service bookings.

Acceptance Criteria

- Customers shall be able to view their profile information.
 - Customers shall be able to update their profile details.
 - Profile information shall include User ID, Full Name, Email Address, Contact Number, and Address.
 - User ID shall be dynamically generated (e.g., U1, U2) and displayed but shall not be editable.
 - Email addresses shall be read-only and remain tied to the profile after creation.
 - Customers shall only access profiles associated with their own User ID.
 - Administrators shall have read-only access to customer profiles.
 - Updated profile information shall be reflected immediately after saving.
 - Mandatory fields shall be validated before saving.
 - Appropriate success, validation, and error messages shall be displayed.
-

US 03 – Vehicle Management

Priority: Must Have

Primary Microservice: User Service

User Story Description

As a **Customer**, I want to manage my vehicles so that I can select the appropriate vehicle while booking a car service.

Acceptance Criteria

- Customers shall be able to add, view, update, and delete vehicles.
 - Each vehicle shall contain Registration Number, Manufacturer, Model, Fuel Type, and Manufacturing Year.
 - Fuel Type shall support the following values: PETROL, DIESEL, CNG, ELECTRIC, HYBRID.
 - Vehicle registration numbers shall be unique.
 - Vehicle ownership shall automatically be associated with the logged-in customer.
 - Customers shall only manage their own vehicles.
 - Administrators shall have read-only access to vehicle information.
 - Vehicles shall support searching, sorting, and pagination.
 - Vehicles linked to active bookings shall not be deleted.
 - Appropriate success, validation, and error messages shall be displayed.
-

US 04 – Service Category Management

Priority: Must Have

Primary Microservice: Catalog Service

User Story Description

As an **Administrator**, I want to manage service categories so that car services can be organized for easy customer navigation

Acceptance Criteria

- Administrator shall be able to add, view, update, and delete service categories.
 - Each category shall contain Category Name and Description.
 - Duplicate category names shall not be allowed.
 - Only Administrator shall manage service categories.
 - Customers shall be able to view all available categories.
 - Categories containing active services shall not be deleted.
 - Mandatory fields shall be validated before saving.
 - Appropriate success, validation, and error messages shall be displayed.
 - Changes to service categories shall be immediately reflected in the application.
-

US 05 – Car Service Management

Priority: Must Have

Primary Microservice: Catalog Service

User Story Description

As an **Administrator**, I want to manage Service Packages so that customers can browse and book available services.

Acceptance Criteria

- Administrator can add, view, update, and delete Service Packages.
- Every service package belongs to a valid service category.
- Each service package includes:
 - Service Name
 - Description
 - Price
 - Availability Status
 - Availability Status supports: ACTIVE, INACTIVE.
- Only Administrator can manage Service Packages.
- Administrator can activate or deactivate services.
- Customers have read-only access to active services.
- Services support searching, sorting, and pagination.
- Services associated with active bookings cannot be deleted.
- Appropriate success, validation, and error messages are displayed.

Trainees must implement the complete Service Package management flow in both frontend and backend, including:

- CRUD/deactivate logic,
- category validation,
- active-booking protection check before delete/deactivate,
- management table rendering and row actions,
- form validation messages and error handling.

US 06 – Browse & Search Services

Priority: Must Have

Primary Microservice: Catalog Service

User Story Description

As a **Customer**, I want to browse and search available Service Packages so that I can choose the most suitable service for my vehicle.

Acceptance Criteria

- Customer can view all available service categories.
- Customer can browse services under each category.
- Customer can search services by Service Name.
- Customer can filter services by Category.
- Customer can sort services by Service Name or Price.
- Service listings support pagination.
- Service details show:
 - Service Name
 - Description
 - Price
 - Availability Status
- Only ACTIVE services are visible to customers.
- Appropriate message is shown when no services are available.
- Admin changes are reflected immediately in customer listings.

Trainees must implement the full browse/search experience across backend and frontend, including:

- active-only retrieval logic,
- category filtering,
- search by name,
- sorting and pagination parameters,
- list rendering and empty-state messaging,
- seamless refresh behavior after admin-side updates.

US 07 – Car Service Booking

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As a **Customer**, I want to book a service appointment for my registered vehicle so that I can schedule maintenance or repair services.

Acceptance Criteria

- Customer can create a booking by selecting:
 - Vehicle
 - Service
 - Appointment Date
 - Time Slot
 - Problem Description (optional)
- Customer can select only vehicles mapped to their own User ID.
- Customer can book only services with ACTIVE availability status.
- Appointment date cannot be in the past.
- Duplicate booking is not allowed for same vehicle + same appointment date + same time slot.
- System auto-generates unique booking reference number in format: BKG + customerId + 000 + bookingCount.
- New booking default status is PENDING.
- Customer gets booking confirmation immediately after successful creation.
- Customer can cancel only bookings in PENDING status.
- Mandatory fields are validated before submission.

Trainees must implement major booking form and flow behavior in both frontend and backend, including:

- form initialization and data binding,
- vehicle/service dropdown population,
- time-slot options,
- field-level validation messages,
- booking submission handling (success + error),
- cancel booking rule (PENDING only).

US 08 – Booking Management

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As an **Administrator**, I want to manage customer bookings so that service appointments can be processed efficiently

Acceptance Criteria

- Administrator shall be able to view all customer bookings.
 - Booking details shall display Customer, Vehicle, Service, Appointment Date, Time Slot, Booking Status, and Problem Description.
 - Administrator shall be able to update booking status.
 - Booking status shall rigidly follow the workflow: PENDING → CONFIRMED → IN_SERVICE → READY_FOR_DELIVERY → COMPLETED (or PENDING → CANCELLED).
 - Invalid booking status transitions shall not be allowed.
 - Bookings shall support searching, sorting, and pagination.
 - Only Administrator shall be authorized to update booking status.
 - Booking list shall display dynamic color-coded UI badges/dropdowns for differing statuses.
 - Updated booking status shall be reflected immediately to both Administrator and Customer.
-

US 09 – Dashboard & Booking History

Priority: Must Have

Primary Microservices: User Service, Service Catalog Service, Booking Service

User Story Description

As a **Customer** or **Administrator**, I want to view dashboard information and booking history so that I can quickly access important information related to my activities.

Acceptance Criteria

- Customers shall be able to view their booking history securely.
 - Customer dashboard shall display summary information including Total Vehicles, Total Bookings, Active Bookings, and Completed Bookings.
 - Administrator dashboard shall display Total Customers, Total Categories, Total Services, and Total Bookings.
 - Administrator shall be able to view recent booking activities.
 - Booking history shall support searching, filtering, sorting, and pagination.
 - Customers shall only access booking history associated with their own User ID.
 - Dashboard metric information shall be retrieved via live cross-service API aggregations.
 - Appropriate messages shall be displayed when no records are available.
 - Newly created or updated bookings shall be reflected immediately in the dashboard and booking history without layout anomalies.
-

US 10 – Booking Search & Reports

Priority: Must Have

Primary Microservice: Booking Service

User Story Description

As an **Administrator**, I want to search and monitor booking records so that I can efficiently manage customer service requests

Acceptance Criteria

- Administrator can search bookings using Booking Reference Number or Customer Name.
- Administrator can filter bookings by Booking Status and Appointment Date.
- Search results support sorting and pagination.
- Booking statistics show total number of bookings grouped by booking status.
- Search and report results are displayed based on logged-in user role.
- Reports table displays:
 - Reference Number
 - Date & Time
 - Status
 - Problem Description
 - (Customer identification is intentionally omitted in this table.)
- Appropriate message is displayed when no matching records are found.
- Appropriate validation and exception handling are implemented.

Trainees must implement both backend and frontend report/search flows, including:

- role-aware search query behavior,
- reference/name/status/date filters,
- pagination and sorting handling,
- status-wise statistics aggregation,
- reports table row rendering and no-results behavior,
- validation/error handling for invalid filters and API failures.

3.1 Final User Story List

User Story	Module	Microservice
US01	Authentication & Authorization	Authentication Service
US02	Customer Profile Management	User Service
US03	Vehicle Management	User Service
US04	Service Category Management	Catalog Service
US05	Service Package Management	Catalog Service
US06	Browse & Search Services	Catalog Service
US07	Car Service Booking	Booking Service
US08	Booking Management	Booking Service

User Story	Module	Microservice
US09	Dashboard & Booking History	Booking, User & Catalog Services
US10	Booking Search & Reports	Booking Service

Data Persistence Rules

The application shall use the respective microservice databases as the permanent storage for all business data.

The following rules shall be followed throughout the application:

- All successful Create, Update, and Delete operations shall be immediately persisted in the corresponding microservice database (auth_db, user_db, catalog_db, booking_db).
- All View and Search operations shall always retrieve the latest persisted records.
- Changes performed by one module shall be immediately reflected wherever the same data is consumed (e.g., dashboard metrics via cross-service API aggregations).
- No business data shall be maintained only in frontend memory; frontend LocalStorage is strictly reserved for managing active session state (user, userId, role).
- Data shall remain available after browser closure, frontend restart, backend restart, application restart, or system reboot.
- Each microservice shall manage only its own database schema, enforcing a strict Database-per-Service pattern.
- The database shall always be considered the single source of truth for all business operations.

Identifier Generation Rules

The system enforces specific identifier rules for business entities. Unlike the previous design, the current implementation utilizes a mix of strict String-based custom identifiers for users/bookings, and standard auto-incrementing numeric IDs for catalog and vehicle entities.

Entity	Format / Data Type	Example	Generation Rule
Administrator	Static String	A1	Seeded statically within the database; not dynamically generated.
Customer	U<number>	U1, U2, U3	Auto-generated by the Auth Service upon registration based on the underlying DB ID.
Booking	BKG<userId>000<count>	BKG10001	Auto-generated by the Booking Service, combining the Customer's User ID and their historical booking count.
Vehicle	Numeric Long	1,2,3	Standard Database Auto-Increment ID. (Business identification relies on the Registration Number).
Service Category	Numeric Long	1,2,3	Standard Database Auto-Increment ID.
Service Package	Numeric Long	1,2,3	Standard Database Auto-Increment ID.

These identifiers shall be generated automatically by the respective microservice during record creation and shall not be editable by end users.

4. Frontend Components

Authentication Module

Handles user registration, login, credential validation, and session management using LocalStorage.

Features:

- User Registration (Customers only)
 - User Login (Customers and Administrators)
 - Role-Based Route Guards
 - Session Management (via LocalStorage, no JWTs)
 - Logout Functionality
-

Dashboard Module

Displays role-based dashboard metric summaries and embedded recent activities.

Features

- Dashboard Summary Cards (Role-Specific)
- Embedded Recent Bookings List
- Quick Navigation
- Responsive Layout
- Dynamic Container Styling (removes borders/margins for embedded views)

Customer Profile Module

Manages customer profile information.

Features

- View Profile
 - Update Profile
 - Form Validation
 - Responsive Forms
-

Vehicle Management Module

Manages customer vehicle information.

Features

- ehicle CRUD (Create, Read, Update, Delete)
- Vehicle List

- Local Search Filtering

Service Package Module

Manages specific car service offerings.

Features

- Service Package CRUD
 - Browse Services
 - Search by Service Name
 - Filter by Category
 - Sorting (by Name or Price)
 - Pagination
 - Availability Status Toggling (ACTIVE / INACTIVE)
-

Car Service Module

Manages car services.

Features

- Service CRUD
 - Browse Services
 - Search
 - Filter
 - Category-wise Listing
 - Sorting
 - Pagination
 - Service Details View
 - Availability Status (ACTIVE / INACTIVE)
-

Booking Module

Manages customer service bookings and administrative workflows.

Features

- Create Booking
- Appointment Date Selection (Future dates only)
- Time Slot Selection
- Booking History
- Booking Cancellation (Only if PENDING)
- View All Bookings (Admin)
- Update Booking Status (Admin)
- Strict Booking Status Workflow Enforcements

- Dynamic Color-Coded Status Badges

Reports Module

Provides booking search, filtering, and statistical reporting functionality.

Features

- Search by Booking Reference Number
- Search by Customer Name (Admin only)
- Filter by Booking Status
- Filter by Appointment Date
- Booking Statistics Aggregation
- Pagination
- Empty State Handling

Shared Core Module

Provides reusable UI components and common application services.

Features

- Global Header
- Dynamic Sidebar (Auto-collapses on navigation)
- Application Footer
- Route Guards
- HTTP Interceptors (Injects X-User-Id and X-User-Role headers)
- Shared UI Components
- Centralized API Services

Backend Components

Authentication Service

Responsibilities

- Registration (Customer profile initialization via User Service)
- Login (Credential validation against Database)
- Header-Based Role Management

User Service

Responsibilities

- Customer Profile
- Vehicle Management

Catalog Service

Responsibilities

- Category Management
- Service Package Management
- Browse and Filter Services

Booking Service

Responsibilities

- Booking Creation (Validates vehicle ownership and active service status)
- Booking Status Management and Workflow
- Dashboard Data Aggregation (Cross-service fetching)
- Reporting and Analytics (Dynamic JPA Specifications)

6. Getting Started

6.1 Setup Steps

1. Start Consul for service discovery.
2. Ensure MySQL is running on port 3306.
3. Start each backend microservice using `mvn clean spring-boot:run` or your IDE.
4. Start the Angular application by running `npm install` followed by `npm start`.
5. Login using the credentials in Section 6.2.

6.2 Default Login Credentials

Role	Email	Password
Admin	admin@gmail.com	admin1234

6.3 Testing Recommendation

Before integrating any feature with Angular, verify the backend endpoint using Postman or Swagger (<http://localhost:<port>/swagger-ui.html>).

7. Environment Requirements

7.1 Technology Stack

Frontend

- Angular 15
- TypeScript
- HTML5
- CSS3
- Bootstrap 5

Backend

- Java 21
- Spring Boot
- Spring Data JPA
- Spring Cloud Gateway
- Spring Cloud Consul
- Spring Cloud Circuit Breaker (Resilience4j)
- REST APIs
- Lombok
- ModelMapper

Database

- MySQL

8. Non-Functional Requirements

- **Performance:** The application should be fast and responsive, with quick load times and minimal lag or delay.
- **Scalability:** The application should be able to handle large amounts of traffic and scale as needed to accommodate growth.
- **Reliability:** The application should be reliable and available, with minimal downtime or outages.
- **Usability:** The application should be easy to use and intuitive, with a clear and consistent user interface.
- **Maintainability:** The application should be easy to maintain and update, with clear and well-organized code that is easy to understand and modify.
- **Compatibility:** The application should be compatible with a wide range of devices and browsers, with support for different screen sizes and resolutions.
- **Implement caching mechanisms** to improve performance.
- **Lazy Loading:** Implement lazy loading techniques for images and other content, ensuring faster initial page loads and smoother user experience.
- **User Interface (UI) and User Experience (UX) Design:** Design an intuitive user interface that is easy to navigate and provides a seamless user experience. Prioritize responsiveness and clarity across all devices (desktop, mobile, tablets).

9. Implementation Guidelines

9.1 General Coding Practices

- Utilize Lombok for model class creation and logging.
- Implement logic using Lambdas and Streams.
- Organize code into logical layers (Controller, Service, Repository, DTO, Entity, Configuration, etc.).
- Use meaningful package names reflecting the domain.
- Employ Spring Dependency Injection for effective component management.

9.2 Error Handling

- Handle cases where the user attempts to access resources without authorization.
- Implement custom exceptions with user-friendly error messages.
- Use a centralized exception handling mechanism to translate exceptions into appropriate HTTP responses.
- *Use `@RestControllerAdvice (GlobalExceptionHandler)` to intercept, format, and log all service exceptions.*
- Return meaningful success and error responses with appropriate HTTP status codes.

9.3 DTOs and Mapping

- Data Transfer Objects (DTOs) must be used for all API request and response payloads.
- Database Entities shall NEVER be exposed directly through REST APIs.
- Use `ModelMapper` to strictly enforce deep conversions between Entities and DTOs within the Service layer.

9.4 Database Interaction

- Spring Data JPA Repositories must be used for all database operations and query generation.
- Configure appropriate database properties and table generation strategies (`spring.jpa.hibernate.ddl-auto=update` for persistence stability).
- Each microservice must strictly maintain its own isolated database schema (e.g., `user_db`, `booking_db`) to ensure loose coupling and prevent cross-domain database querying.

9.5 API Design

- All APIs should be mapped with an appropriate base URI representing the resource collection (e.g., `/api/vehicles`).
- Ensure strict isolation of server port numbers for each microservice (e.g., Gateway: 8080, Auth: 8081, User: 8082, Catalog: 8083, Booking: 8084).
- Use appropriate HTTP methods (GET, POST, PUT, DELETE) and let Spring automatically resolve appropriate status codes.

9.6 Microservice Communication

- Register all microservices with Spring Cloud Consul for dynamic service discovery and health monitoring.
- Route all frontend and external client requests exclusively through the Spring Cloud Gateway.
- Use `RestTemplate` for synchronous inter-service communication via HTTP GET/POST calls.
- Implement Spring Cloud Circuit Breaker (`Resilience4j`) for inter-service calls using the following precise configuration:

- slidingWindowSize: 10 (Evaluates the last 10 calls)
- failureRateThreshold: 50% (Opens the circuit if 50% of calls fail)
- waitDurationInOpenState: 60 seconds (Time before transitioning to half-open state)
- Define appropriate @CircuitBreaker fallback methods to ensure graceful degradation

9.7 Validation

- Bean Validation should be used for all user inputs.
- Custom Validators should be implemented for complex validations.
- All request parameters must be validated for null or empty values. *Return standard Bean Validation error messages.*

9.8 Testing

- Write unit test cases for critical service methods using **JUnit** and **Mockito**.
- Achieve a minimum of 80% code coverage.
- Test all REST APIs using Postman or Swagger/OpenAPI.

10. Code Quality

To ensure adherence to coding standards, the project should be analyzed using SonarQube. The following minimum acceptable values must be met:

SonarQube Metric	Minimum Acceptable Value
Security	A
Reliability	A
Issues	≤ 5
Coverage	≥ 80%
Duplications	≤ 3%
Security Hotspots	A

10.1 REST API Testing

Once the implementation is completed, all REST endpoints should be tested using Postman or Swagger/OpenAPI to verify that the application functions correctly.

11. Final Deliverables

- Application archive (.jar) with source code
- Database DDL Script
- Complete Source Code

- Sample screenshots of important screens

Good Luck! Build clean, maintainable, and well-tested code.